

4. 管理者・投稿管理編

対象範囲

実装する機能

機能	実装内容
管理者権限確認	一般ユーザーによる管理者投稿APIの利用を拒否する
投稿一覧	未削除の投稿を管理者向けに一覧表示する
投稿詳細	投稿情報、投稿者、状態、確認可能な変換後動画を表示する
投稿の非公開	ready / published を ready / hidden へ変更する
投稿の公開再開	ready / hidden を ready / published へ変更する
監査記録	非公開・公開再開の理由、操作者、対象、変更前後、Request IDを保存する
管理者画面	投稿一覧、詳細、非公開、公開再開をブラウザから操作する
OpenAPI	管理者投稿APIの契約を固定する
テスト	Entity、Repository、Usecase、Controller、Frontend、結合、ブラウザ確認

前提条件

本編の実装前に、次が完成していること。

前提	使用目的
User Entity	管理者Roleと投稿者Statusを確認する
Video Entity	処理状態、公開状態、削除状態を管理する
AdminAuditLog Entity	管理操作を記録する
usersテーブル	投稿者状態を確認する
videosテーブル	投稿情報と状態を取得・更新する
video_output metasテーブル	管理者確認用の変換後動画を取得する
admin_audit_logsテーブル	非公開・公開再開の監査記録を保存する
Access Token	管理者API認証に使用する
Auth Middleware	DB上のUser状態とTokenVersionを確認する

前提	使用目的
Admin Middleware	Roleが <code>admin</code> であることを確認する
AuthContext	FrontendでUserとRoleを共有する
AdminRoute	一般ユーザーへ管理者画面を表示しない
ObjectStorageRepository	期限付き再生URLとサムネイルURLを発行する
Request ID	APIログと監査ログを紐づける
共通エラー形式	管理者投稿APIでも同じエラー形式を使用する

既存の管理者・ユーザー管理編では、`AdminRoute`、Admin Middleware、管理者APIの認証方式、Cursorページング、監査ログの基本構造が定義済みである。本編ではそれらを再実装せず再利用する。

要件上の機能順

要件定義に記載された順番を維持する。

1. 投稿一覧・詳細確認
2. 管理者アカウントから投稿の非公開
3. 管理者アカウントから投稿の公開再開

実際のコード実装順

Backend

1. 既存の `Video`、`AdminAuditLog`、Admin Middlewareを確認する。
2. `Video.HideByAdmin` と `Video.RestoreByAdmin` がEntityへ存在することを確認する。
3. `AdminAuditLog` へ `video_hide` と `video_restore` が定義されていることを確認する。
4. `videos` と `admin_audit_logs` のGo Migrationを確認する。
5. `admin_audit_logs` のCHECK制約へ `video_hide` と `video_restore` が含まれることを確認する。
6. `idx_videos_admin_list` が存在することを確認する。
7. 管理者投稿用Domain Errorを確認・追加する。
8. `AdminVideoRepository` Interfaceを定義する。

9. `AdminVideoRepository` をGORMで実装する。
10. 投稿一覧のCursorページングを実装する。
11. 投稿詳細取得を実装する。
12. 非公開Transactionを実装する。
13. 公開再開Transactionを実装する。
14. `AdminVideoUsecase` を実装する。
15. `AdminVideoValidator` を実装する。
16. `AdminVideoController` を実装する。
17. Routerへ管理者投稿APIを接続する。
18. `main.go` でRepository、Usecase、Validator、Controllerを接続する。
19. OpenAPIへ管理者投稿APIを追加する。
20. Backendテストを実行する。
21. 管理者投稿APIを確認する。

Frontend

1. 管理者投稿用Typeを作成する。
 2. 管理者投稿API Clientを作成する。
 3. 投稿一覧画面を作成する。
 4. 投稿詳細画面を作成する。
 5. 非公開理由入力と非公開操作を接続する。
 6. 公開再開理由入力と公開再開操作を接続する。
 7. 既存の `AdminRoute` へ画面を接続する。
 8. 管理者画面のナビゲーションへ「投稿管理」を追加する。
 9. Frontendテストを実行する。
 10. BackendとFrontendを起動する。
 11. ブラウザで管理者投稿管理フローを確認する。
-

完成状態

本編完成後、次を確認できること。

1. activeな管理者だけが管理者投稿APIを利用できる。
 2. 一般ユーザーが管理者投稿APIを呼ぶと `403 Forbidden` になる。
 3. 未認証状態では管理者投稿APIを利用できない。
 4. 未削除の投稿が管理者一覧へ表示される。
 5. 投稿は `CreatedAt` 降順、同日時の場合は `ID` 降順で並ぶ。
 6. 投稿者、タイトル、説明、処理状態、公開状態、投稿日時を確認できる。
 7. `ready / published` 動画だけを管理者が非公開にできる。
 8. 非公開理由が必須となる。
 9. 非公開操作と監査ログ作成が同じTransactionで確定する。
 10. 非公開後は `ready / hidden` になる。
 11. `hidden` 動画がリールへ表示されない。
 12. `hidden` 動画が保存一覧へ表示されない。
 13. 後続の動画検索でも `hidden` 動画が表示されない。
 14. 投稿者が `hidden` 動画を再公開できない。
 15. 投稿者へ `hidden` 動画の再生URLを返さない。
 16. activeな管理者は管理者詳細画面で `hidden` 動画を確認できる。
 17. `ready / hidden` 動画だけを管理者が公開再開できる。
 18. 投稿者がactiveの場合だけ公開再開できる。
 19. 投稿者がsuspendedの場合は公開再開できない。
 20. 公開再開理由が必須となる。
 21. 公開再開操作と監査ログ作成が同じTransactionで確定する。
 22. 公開再開後は `ready / published` になる。
 23. 一覧、詳細、非公開、公開再開でObject Keyを返さない。
 24. SQL、DB制約名、Stack Trace、Storage Credentialを返さない。
 25. 同時操作によって不正な状態へ遷移しない。
- Videoの管理者操作は `ready / published → ready / hidden`、`ready / hidden → ready / published` へ限定されている。投稿者がsuspendedの場合の公開再開は禁止される。

全体フロー

投稿一覧

1. 管理者が管理者投稿一覧画面を開く。
2. Frontendが `GET /admin/videos` を呼ぶ。
3. Auth MiddlewareがAccess Tokenを検証する。
4. Auth MiddlewareがUserのStatusとTokenVersionをDBで確認する。
5. Admin MiddlewareがRoleを確認する。
6. Controllerが `limit` と `cursor` を受け取る。
7. ValidatorがQueryを検証する。
8. UsecaseがRepositoryへ一覧取得を依頼する。
9. Repositoryが未削除のVideoと投稿者を取得する。
10. `CreatedAt` 降順、`ID` 降順で取得する。
11. Usecaseが次ページCursorを生成する。
12. Controllerが一覧Responseを返す。
13. Frontendが投稿一覧を表示する。

投稿詳細

1. 管理者が投稿を選択する。
2. Frontendが `GET /admin/videos/:video_id` を呼ぶ。
3. Auth MiddlewareとAdmin Middlewareが管理者を確認する。
4. ValidatorがVideo IDを検証する。
5. RepositoryがVideo、投稿者、OutputMetaを取得する。
6. 削除済みVideoの場合は `404 Not Found` を返す。
7. `ready / published` または `ready / hidden` でOutputMetaが存在する場合だけ、Usecaseが期限付きURLを発行する。
8. `private` 動画へ管理者用再生URLを発行しない。
9. Controllerが投稿詳細を返す。
10. Frontendが投稿情報と確認可能な動画を表示する。

管理者による非公開

1. 管理者が `ready / published` 動画の詳細を開く。
2. 管理者が非公開理由を入力する。
3. Frontendが `PATCH /admin/videos/:video_id/hide` を呼ぶ。
4. ControllerがVideo ID、Reason、Request ID、管理者Userを取得する。
5. ValidatorがVideo IDとReasonを検証する。
6. Usecaseが管理者のRoleとStatusを再確認する。
7. RepositoryがDB Transactionを開始する。
8. RepositoryがVideoを行ロックする。
9. `ready / published / 未削除` であることを再確認する。
10. Entityの `HideByAdmin` を実行する。
11. PublishStatusを `hidden` へ変更する。
12. UpdatedAtを更新する。
13. `video_hide` のAdminAuditLogを作成する。
14. Video更新と監査ログ作成が成功した場合だけCommitする。
15. Controllerが更新後の状態を返す。
16. Frontendが詳細を再取得する。
17. 公開画面から対象動画が消える。

管理者による公開再開

1. 管理者が `ready / hidden` 動画の詳細を開く。
2. 管理者が公開再開理由を入力する。
3. Frontendが `PATCH /admin/videos/:video_id/restore` を呼ぶ。
4. ControllerがVideo ID、Reason、Request ID、管理者Userを取得する。
5. ValidatorがVideo IDとReasonを検証する。
6. RepositoryがDB Transactionを開始する。
7. RepositoryがVideoを行ロックする。
8. `ready / hidden / 未削除` であることを再確認する。

9. 投稿者の現在Statusを取得する。
10. 投稿者がsuspendedの場合は公開再開を拒否する。
11. Entityの `RestoreByAdmin` を実行する。
12. PublishStatusを `published` へ変更する。
13. UpdatedAtを更新する。
14. `video_restore` のAdminAuditLogを作成する。
15. Video更新と監査ログ作成が成功した場合だけCommitする。
16. Controllerが更新後の状態を返す。
17. Frontendが詳細を再取得する。
18. 公開条件を満たす動画が再びリールへ表示される。

Clean Architectureの責務

層	責務
Entity	Videoの管理者用状態遷移、AdminAuditLogの不変条件を保持する
DB	Video、User、OutputMeta、監査ログを永続化する
Repository	GORM、一覧Query、行ロック、Transaction、監査ログ保存を扱う
Usecase	管理者が投稿へ行う操作の業務フローを組み立てる
Validator	Video ID、Reason、limit、cursorを検証する
Controller	Echo Context、Path、Query、JSON、Request ID、HTTP Responseを扱う
Middleware	認証済みでactiveなadminであることを確認する
Router	URL、HTTPメソッド、Middleware、Controllerを接続する
ObjectStorageRepository	許可済みOutput Objectの期限付き取得URLを発行する
Frontend API	管理者投稿APIへの通信をまとめる
AdminRoute	一般ユーザーに管理画面を表示しない
React Page	投稿一覧、詳細、非公開、公開再開を表示・操作する

禁止事項

- Entityへgorm.DB、GORMによるQuery、行ロック、Transaction、Echo Contextを持ち込まない。DBマッピングに必要な最小限のGORMタグと、JSON出力制御に必要なJSONタグはEntityへ記載。
- ControllerでDBを検索しない。
- Controllerで状態遷移を行わない。
- Usecaseへ `gorm.DB` を渡さない。
- UsecaseでBegin、Commit、Rollbackを扱わない。
- 汎用的な `TxManager` を作らない。
- RepositoryからEcho用Responseを返さない。
- Routerへ業務処理を書かない。
- 管理者画面の表示制御だけを認可として使用しない。
- `any` や `map[string]interface{}` を使用しない。
- User入力をSQLやORDER BYへ直接連結しない。

Transactionの方針

共通方針

汎用的な `TxManager` は作成しない。

管理者による非公開と公開再開は、それぞれ `AdminVideoRepository` の用途別メソッド内で完結させる。

処理	Transaction境界
管理者非公開	<code>AdminVideoRepository.Hide</code>
管理者公開再開	<code>AdminVideoRepository.Restore</code>

管理者非公開Transaction

同じTransaction内で次を行う。

1. Videoを `FOR UPDATE` で取得する。
2. Videoが未削除であることを確認する。
3. ProcessingStatusが `ready` であることを確認する。

4. PublishStatusが `published` であることを確認する。
5. Entityの `HideByAdmin` を実行する。
6. PublishStatusとUpdatedAtを更新する。
7. `video_hide` のAdminAuditLogを作成する。
8. 全処理成功時だけCommitする。
9. 途中失敗時はRollbackする。

監査ログ作成に失敗した状態でVideoだけを `hidden` にしない。

管理者公開再開Transaction

同じTransaction内で次を行う。

1. Videoを `FOR UPDATE` で取得する。
2. Videoが未削除であることを確認する。
3. ProcessingStatusが `ready` であることを確認する。
4. PublishStatusが `hidden` であることを確認する。
5. 投稿者の現在Statusを取得する。
6. 投稿者がactiveであることを確認する。
7. Entityの `RestoreByAdmin` を実行する。
8. PublishStatusとUpdatedAtを更新する。
9. `video_restore` のAdminAuditLogを作成する。
10. 全処理成功時だけCommitする。
11. 途中失敗時はRollbackする。

ロック順

管理者の単一動画操作はVideoだけを行ロックする。

投稿者Userは状態確認のために取得するが、管理者公開再開TransactionではUser行をロックしない。

理由は次のとおり。

- User停止Transactionは `users → refresh_tokens → videos` の順でロックする。
- 公開再開処理が `videos → users` の順でロックするとデッドロックの原因になる。

- 同時にUser停止が実行された場合、停止Transactionが最終的に公開動画を `hidden` へ戻すため、安全側の状態になる。

DB仕様では、管理者の単一動画操作は `videos` だけをロックし、管理者非公開・公開再開のUPDATE条件へ現在状態と `deleted_at IS NULL` を含める方針である。

Transactionへ含めない処理

次をDB Transaction中に実行しない。

- Object Storageへの通信
- 期限付き再生URL発行
- 期限付きサムネイルURL発行
- Frontend通知
- 外部HTTP通信

Storage応答待ちの間にVideo行ロックを保持しない。

ファイル構成

Backend

```
backend/
├── entity/
│   ├── user.go
│   ├── video.go
│   ├── admin_audit_log.go
│   └── errors.go
├── db/
│   └── db.go
├── migrate/
│   ├── users.go
│   ├── videos.go
│   ├── video_output metas.go
│   └── admin_audit_logs.go
├── repository/
│   ├── object_storage_repository.go
│   ├── admin_user_repository.go
│   └── admin_video_repository.go
├── usecase/
│   └── admin_user_usecase.go
```

```
|   └─ admin_video_usecase.go
|
|   └─ validator/
|       │   └─ admin_user_validator.go
|       │   └─ admin_video_validator.go
|
|   └─ controller/
|       │   └─ admin_user_controller.go
|       │   └─ admin_video_controller.go
|
|   └─ middleware/
|       │   └─ auth_middleware.go
|       │   └─ admin_middleware.go
|
|   └─ router/
|       │   └─ router.go
|
|   └─ main.go
```

Frontend

```
frontend/
└─ src/
    │   └─ api/
    │       │   └─ client.ts
    │       │   └─ admin_user.ts
    │       │   └─ admin_video.ts
    │
    │   └─ auth/
    │       │   └─ AdminRoute.tsx
    │       └─ AuthProvider.tsx
    │       └─ ProtectedRoute.tsx
    │       └─ authContext.ts
    │       └─ tokenStore.ts
    │       └─ useAuth.ts
    │
    │   └─ components/
    │       │   └─ VideoStatusBadge.tsx
    │
    │   └─ pages/
    │       │   └─ AdminUsersPage.tsx
    │       │   └─ AdminUserDetailPage.tsx
    │       │   └─ AdminVideosPage.tsx
    │       │   └─ AdminVideoDetailPage.tsx
    │
    │   └─ router/
    │       │   └─ AppRouter.tsx
    │
    │   └─ types/
    │       │   └─ admin_user.ts
    │       │   └─ admin_video.ts
```

新しい管理者ダッシュボードを不要に追加しない。

既存の管理者用導線へ「ユーザー管理」と「投稿管理」のリンクを配置する。

同じ画面でしか使わないModalや処理を不要に別Hookへ分割しない。

Entity仕様

Video

既存のVideo Entityを使用する。新しい管理者投稿Entityは作らない。

使用する状態

項目	値
処理完了	<code>ready</code>
一般公開	<code>published</code>
管理者非公開	<code>hidden</code>
投稿者非公開	<code>private</code>

使用する操作

操作	事前条件	状態変更	失敗条件
<code>HideByAdmin(now time.Time) error</code>	<code>ready</code> 、 <code>published</code> 、未削除	<code>hidden</code> へ変更し、UpdatedAtを更新	処理未完了、既に非公開、削除済み
<code>RestoreByAdmin(ownerActive bool, now time.Time) error</code>	<code>ready</code> 、 <code>hidden</code> 、未削除、投稿者active	<code>published</code> へ変更し、UpdatedAtを更新	投稿者停止中、状態不正、削除済み
<code>CanBeViewedPublicly() bool</code>	なし	なし	<code>ready</code> / <code>published</code> / 未削除 だけtrue
<code>RepublishByOwner(...) error</code>	投稿者本人、 <code>ready</code> / <code>private</code>	<code>published</code> へ変更	<code>hidden</code> の場合は拒否

既存Entity仕様では、管理者非公開・公開再開用の `HideByAdmin` と `RestoreByAdmin` が定義され、投稿者による `hidden` 解除は禁止されている。

守るルール

- 管理者は `private` 動画を `hidden` へ変更しない。
- 管理者は処理中動画を `hidden` へ変更しない。
- 管理者は `failed` 動画を `hidden` へ変更しない。
- 管理者は削除済み動画を操作しない。
- 非公開時にProcessingStatusを変更しない。
- 公開再開時にProcessingStatusを変更しない。
- 非公開時に動画ファイルを削除しない。
- 非公開時にSavedVideoやLikeを削除しない。
- 公開再開時にSavedVideoやLikeを再作成しない。
- Entityは非公開理由やRequest IDを保持しない。
- 理由とRequest IDはAdminAuditLogへ保存する。

AdminAuditLog

既存のAdminAuditLogを使用する。

使用するAction

Action	TargetType	BeforeStatus	AfterStatus
<code>video_hide</code>	<code>video</code>	<code>published</code>	<code>hidden</code>
<code>video_restore</code>	<code>video</code>	<code>hidden</code>	<code>published</code>

保存するフィールド

フィールド	仕様
ID	DBが発行する監査ログID
AdminUserID	操作した管理者ID
TargetType	<code>video</code>
TargetID	対象Video ID
Action	<code>video_hide</code> または <code>video_restore</code>

フィールド	仕様
BeforeStatus	操作前のPublishStatus
AfterStatus	操作後のPublishStatus
Reason	trim後1～500文字
RequestID	APIログと紐づけるRequest ID
CreatedAt	操作成功日時

不変条件

- AdminUserIDを0にしない。
- TargetIDを0にしない。
- Reasonを空文字にしない。
- RequestIDを空文字にしない。
- BeforeStatusとAfterStatusを同じ値にしない。
- ActionとTargetTypeと変更状態を一致させる。
- 作成後に更新しない。
- 作成後に削除しない。
- Password、Token、Cookie、秘密鍵、署名付きURL、Object Keyを保存しない。
- Video状態変更と同じTransaction内で作成する。

AdminAuditLogの正本では、`video_hide` と `video_restore` が管理者投稿操作として定義されている。

DB仕様

使用するテーブル

テーブル	用途
users	投稿者ID、名前、Statusを確認する
videos	投稿情報、処理状態、公開状態、削除状態を取得・更新する
video_output metas	変換後動画とサムネイルのObject Keyを内部取得する
admin_audit_logs	非公開・公開再開の監査記録を保存する

テーブル	用途
saved_videos	状態変更時は更新せず、公開取得条件で除外する
video_likes	状態変更時は更新せず、公開取得条件で新規操作を拒否する

Migration

MigrationはGoで実装する。

通常のAPI起動時に無条件実行しない。

本編では原則として新しいテーブルを作成しない。

次を確認する。

videos

- `processing_status` へ `ready` が許可されている。
- `publish_status` へ `published` と `hidden` が許可されている。
- `hidden` は `ready` かつ未削除の場合だけ保存できる。
- `idx_videos_admin_list` が存在する。

admin_audit_logs

CHECK制約に次が含まれていること。

Action	Target	Before	After
<code>video_hide</code>	<code>video</code>	<code>published</code>	<code>hidden</code>
<code>video_restore</code>	<code>video</code>	<code>hidden</code>	<code>published</code>

既に適用済みのMigrationで2つのActionが不足している場合、既存Migrationを書き換えない。

新しいGo Migrationで監査ログのCHECK制約を置き換える。

本編のDB仕様では、`admin_audit_logs`へ`video_hide`と`video_restore`の整合性保証を定義し、`videos`には管理者一覧用Indexを定義する。

一覧取得条件

管理者投稿一覧では、未削除の投稿を取得する。

```
deleted_at IS NULL
```

次の全処理状態を一覧対象とする。

- `uploading`
- `expired`
- `uploaded`
- `processing`
- `ready`
- `failed`

次の全公開状態を一覧対象とする。

- `private`
- `published`
- `hidden`

管理者一覧は公開一覧ではないため、`ready / published` だけに限定しない。

管理者自身が投稿したVideoも一覧対象とする。

並び順

1. `created_at DESC`
2. `id DESC`

Cursor条件

Cursorがある場合は次の条件を使用する。

```
created_at < cursor.created_at
OR (
  created_at = cursor.created_at
  AND id < cursor.id
)
```

ORDER BYへUser入力を連結しない。

Cursor値はプレースホルダーで渡す。

状態更新条件

操作	UPDATE条件
管理者非公開	<code>id = ? AND processing_status = 'ready' AND publish_status = 'published' AND deleted_at IS NULL</code>
管理者公開再開	<code>id = ? AND processing_status = 'ready' AND publish_status = 'hidden' AND deleted_at IS NULL</code>

更新件数が0の場合は対象を再取得し、次を区別する。

- 存在しない・削除済み
- 現在状態が操作条件と異なる
- 投稿者がsuspended
- 同時更新による競合

Repository仕様

共通方針

- InterfaceをUsecaseへ注入する。
- GORMはRepository内だけで扱う。
- GORM ModelをAPI Responseとして返さない。
- User入力をSQLへ直接連結しない。
- ORDER BYを固定する。
- `any` を使用しない。
- `map[string]interface{}` を使用しない。
- 汎用 `TxManager` を作成しない。
- Transactionは用途別Repositoryメソッドで完結させる。
- DBエラーをDomain Errorへ変換する。
- SQLSTATEや制約名をAPIへ返さない。

AdminVideoRepository

メソッド	責務
<code>List</code>	未削除投稿をCursor方式で取得する
<code>FindByID</code>	管理者用投稿詳細を取得する

メソッド	責務
<code>Hide</code>	Video非公開と監査ログ作成を同一Transactionで行う
<code>Restore</code>	Video公開再開と監査ログ作成を同一Transactionで行う

List

Repositoryは次を返す固定構造体を用意する。

項目	内容
VideoID	Video ID
AuthorID	投稿者User ID
AuthorName	投稿者名
AuthorStatus	<code>active</code> または <code>suspended</code>
Title	タイトル
Description	説明
Category	カテゴリー
ProcessingStatus	処理状態
PublishStatus	公開状態
CreatedAt	投稿日時
UpdatedAt	更新日時

Repositoryは `limit + 1` 件取得する。

結果	処理
取得件数がlimit以下	<code>HasMore = false</code>
取得件数がlimit + 1	最後の1件をResponse対象から外し、 <code>HasMore = true</code>
0件	空配列、 <code>HasMore = false</code>

RepositoryはBase64 Cursorを生成しない。

RepositoryはCursor構造体の値を使用したDB取得だけを担当する。

FindByID

次を取得する。

- Video
- 投稿者ID

- 投稿者名
- 投稿者Status
- OutputMeta

削除済みVideoは取得対象外とする。

元動画Object Keyは管理者用Read Modelへ含めない。

OutputMetaのObject KeyはUsecaseが期限付きURLを発行するための内部値としてだけ返す。

Controller用Responseへそのまま渡さない。

Hide

同じTransactionで次を行う。

1. Videoを行ロックする。
2. 現在状態を確認する。
3. Entityの `HideByAdmin` を呼ぶ。
4. PublishStatusとUpdatedAtを保存する。
5. AdminAuditLogを作成する。
6. Commitする。

Restore

同じTransactionで次を行う。

1. Videoを行ロックする。
2. 現在状態を確認する。
3. 投稿者Statusを取得する。
4. Entityの `RestoreByAdmin` を呼ぶ。
5. PublishStatusとUpdatedAtを保存する。
6. AdminAuditLogを作成する。
7. Commitする。

ObjectStorageRepository

既存の `CreateReadURL` を使用する。

条件	発行対象
<code>ready / published / 未削除</code>	変換後動画、サムネイル
<code>ready / hidden / 未削除</code>	管理者詳細APIだけ変換後動画、サムネイル
<code>ready / private</code>	管理者には発行しない
処理未完了	発行しない
failed	発行しない
削除済み	発行しない

一覧APIでは期限付き動画URLを発行しない。

投稿詳細APIを開いた場合だけ発行する。

Usecase仕様

AdminVideoUsecase

メソッド	内容
<code>List</code>	管理者投稿一覧を取得する
<code>Get</code>	管理者投稿詳細を取得する
<code>Hide</code>	理由付きで投稿を非公開にする
<code>Restore</code>	理由付きで投稿を公開再開する

依存するInterface

- `AdminVideoRepository`
- `ObjectStorageRepository`

Usecaseは次を扱わない。

- Echo Context
- HTTP Status
- Cookie
- Response Header
- GORM

- DB TransactionのBegin、Commit、Rollback
- Storage SDK固有型
- Base64を含むHTTP Query取得
- React画面状態

List

1. 認証済みUserを受け取る。
2. Userがactiveなadminであることを確認する。
3. 検証済みlimitとCursorを受け取る。
4. Repositoryの `List` を呼ぶ。
5. Repository結果をUsecase出力へ変換する。
6. `HasMore = true` の場合、Responseへ含める最後のVideoからCursorを生成する。
7. CursorのCreatedAtをUTCへ変換する。
8. Cursorを固定構造体へ保存する。
9. JSON化する。
10. Base64 Raw URL Encodingで文字列化する。
11. 一覧結果を返す。

Cursorへ次だけを含める。

項目	型
<code>created_at</code>	UTC日時
<code>id</code>	uint64

Cursorを認証情報として使用しない。

Get

1. 認証済みUserを受け取る。
2. Userがactiveなadminであることを確認する。
3. 検証済みVideo IDを受け取る。
4. Repositoryの `FindByID` を呼ぶ。
5. Videoが存在することを確認する。

6. `ready / published` または `ready / hidden` か確認する。
7. OutputMetaがある場合、変換後動画の期限付きURLを発行する。
8. サムネイルの期限付きURLを発行する。
9. `private` 動画にはURLを発行しない。
10. Object Keyを除外した結果を返す。

`ready / published` または `ready / hidden` なのにOutputMetaがない場合は、データ整合性エラーとして扱う。

Hide

1. 認証済みUserを受け取る。
2. Userがactiveなadminであることを確認する。
3. 検証済みVideo IDを受け取る。
4. trim済みReasonを受け取る。
5. Request IDを受け取る。
6. 現在時刻をUTCで取得する。
7. Repositoryの `Hide` を呼ぶ。
8. 更新後のVideo状態を返す。

非公開後に次を行わない。

- Storage Object削除
- SavedVideo削除
- VideoLike削除
- 投稿者Token失効
- 投稿者Status変更

Restore

1. 認証済みUserを受け取る。
2. Userがactiveなadminであることを確認する。
3. 検証済みVideo IDを受け取る。
4. trim済みReasonを受け取る。

5. Request IDを受け取る。
6. 現在時刻をUTCで取得する。
7. Repositoryの `Restore` を呼ぶ。
8. 更新後のVideo状態を返す。

公開再開は、非公開原因に関係なく同じ操作を使用する。

hiddenになった原因	公開再開
管理者が個別非公開	投稿者activeなら可能
User利用停止に伴う非公開	User再開後、管理者が個別に実行可能
投稿者が現在suspended	不可

User利用再開時に自動公開しない。

Validator仕様

AdminVideoValidator

メソッド	検証対象
<code>ValidateVideoID</code>	Path Video ID
<code>ValidateReason</code>	非公開・公開再開理由
<code>ValidateListQuery</code>	limit、cursor

Video ID

- 数字であること
- 1以上
- `math.MaxInt64` 以下
- 負数を拒否する
- 0を拒否する
- 桁あふれを拒否する

Reason

項目	条件
必須	必須
正規化	前後空白を除去する
最小	1文字
最大	500文字
空白だけ	拒否する
不明JSONフィールド	拒否する

Reasonは監査ログとして保存する。

HTMLとして実行する目的の変換は行わない。

Reactでは通常の文字列として表示する。

limit

条件	値
未指定	20
最小	1
最大	100
数字以外	400

cursor

Cursor未指定は最初のページとする。

検証手順は次のとおり。

1. Base64 Raw URL EncodingとしてDecodeする。
2. 専用Cursor構造体へJSON Decodeする。
3. 不明フィールドを拒否する。
4. CreatedAtが有効な日時であることを確認する。
5. CreatedAtをUTCへ変換する。
6. IDが1以上であることを確認する。
7. IDが `math.MaxInt64` 以下であることを確認する。
8. 不正なCursorは `400 Bad Request` とする。

`any` や `map[string]interface{}` でDecodeしない。

ValidatorはCursor内のVideoがDBに存在するか確認しない。

Controller仕様

AdminVideoController

メソッド	内容
List	管理者投稿一覧を返す
Detail	管理者投稿詳細を返す
Hide	管理者非公開を実行する
Restore	管理者公開再開を実行する

Controllerは次を行わない。

- Role判定の最終決定
- Video状態遷移
- 投稿者Status判定
- DB操作
- 行ロック
- Transaction制御
- 監査ログの直接保存
- Object Key取得
- Storage SDK操作
- SQL生成
- Cursor取得用SQL生成

List

1. Contextから認証Userを取得する。
2. Queryからlimitとcursorを取得する。
3. Validatorを呼ぶ。
4. 検証済み入力をUsecaseへ渡す。
5. Usecase結果をResponseへ変換する。

6. 200 OK を返す。

Detail

1. Contextから認証Userを取得する。
2. PathからVideo IDを取得する。
3. Validatorを呼ぶ。
4. Usecaseへ渡す。
5. Object Keyを含まないResponseへ変換する。
6. 200 OK を返す。

Hide

1. Contextから認証Userを取得する。
2. ContextからRequest IDを取得する。
3. PathからVideo IDを取得する。
4. JSONを専用Request型へBindする。
5. 不明フィールドを拒否する。
6. ValidatorでVideo IDとReasonを検証する。
7. Usecaseへ渡す。
8. 更新後状態を返す。
9. 200 OK を返す。

Restore

1. Contextから認証Userを取得する。
2. ContextからRequest IDを取得する。
3. PathからVideo IDを取得する。
4. JSONを専用Request型へBindする。
5. ValidatorでVideo IDとReasonを検証する。
6. Usecaseへ渡す。
7. 更新後状態を返す。

8. `200 OK` を返す。

Request IDがContextに存在しない場合、空文字の監査ログを作成せず内部設定エラーとして扱う。

Middleware仕様

Auth Middleware

全管理者投稿APIへ適用する。

次を確認する。

- Authorization Headerが存在する
- Bearer Access Tokenである
- JWT署名が正しい
- Access Tokenが期限内
- UserがDBに存在する
- User Statusがactive
- JWT内TokenVersionとDBのTokenVersionが一致する

Admin Middleware

Auth Middlewareの後に実行する。

次を確認する。

- ContextにUserが存在する
- User Roleが `admin`
- User Statusが `active`

一般ユーザーは `403 Forbidden` とする。

FrontendのAdminRouteは画面表示制御であり、BackendのAdmin Middlewareを代替しない。

Rate Limit

要件定義では管理者投稿一覧、詳細、非公開、公開再開に対する個別Rate Limitは定義されていない。

本編では新しいRate Limitを勝手に追加しない。

将来追加する場合は、要件定義へ次を追加してから実装する。

- 対象API
- 識別単位
- 最大Token数
- 補充速度
- Redis Key
- 超過時の挙動

CSRF

管理者投稿APIはAuthorization HeaderのAccess Tokenを使用する。

Refresh Token Cookieを認証へ使用しないため、CSRF Middlewareを適用しない。

Body Limit

非公開・公開再開Requestは小さいJSONだけを受け取る。

既存の共通JSON Body Limitを適用する。

Router・API仕様

API一覧

HTTP	URL	内容	認証
GET	/admin/videos	管理者投稿一覧	Auth、Admin
GET	/admin/videos/:video_id	管理者投稿詳細	Auth、Admin
PATCH	/admin/videos/:video_id/hide	管理者非公開	Auth、Admin
PATCH	/admin/videos/:video_id/restore	管理者公開再開	Auth、Admin

Middleware接続順

1. Request ID
2. Recover
3. Logger

4. Security Header
5. CORS
6. Body Limit
7. Auth Middleware
8. Admin Middleware
9. Controller

成功Status

API	Status
投稿一覧	200 OK
投稿詳細	200 OK
投稿非公開	200 OK
投稿公開再開	200 OK

API Request ・ Response

投稿一覧

Request

```
GET /admin/videos?limit=20
```

次ページ：

```
GET /admin/videos?limit=20&cursor={next_cursor}
```

FrontendはCursorを解析・変更しない。

Response

```
{
  "items": [
    {
      "id": 125,
```

```

    "author": {
      "id": 10,
      "name": "山田 太郎",
      "status": "active"
    },
    "title": "ハンドドリップの蒸らし方",
    "description": "蒸らし時間の違いを紹介します",
    "category": "brewing",
    "processing_status": "ready",
    "publish_status": "published",
    "created_at": "2026-07-16T11:50:00Z",
    "updated_at": "2026-07-16T11:55:00Z"
  }
],
"next_cursor": null,
"has_more": false
}

```

一覧Responseルール

- `items` は0件でも `null` ではなく空配列とする。
- 次ページがない場合は `next_cursor = null` とする。
- 次ページがない場合は `has_more = false` とする。
- `total` を返さない。
- 一覧取得で `COUNT(*)` を実行しない。
- 一覧で再生URLを発行しない。
- 一覧でObject Keyを返さない。
- 削除済みVideoを返さない。

投稿詳細

Response

```

{
  "id": 125,
  "author": {
    "id": 10,
    "name": "山田 太郎",
    "status": "active"
  },
  "title": "ハンドドリップの蒸らし方",
  "description": "蒸らし時間の違いを紹介します",
  "category": "brewing",
  "processing_status": "ready",

```

```
"publish_status":"hidden",
"playback_url":"期限付き再生URL",
"thumbnail_url":"期限付きサムネイルURL",
"created_at":"2026-07-16T11:50:00Z",
"updated_at":"2026-07-16T12:10:00Z"
}
```

URL項目

Video状態	playback_url	thumbnail_url
ready / published	期限付きURL	期限付きURL
ready / hidden	管理者詳細だけ期限付きURL	管理者詳細だけ期限付きURL
ready / private	null	null
処理中	null	null
failed	null	null
deleted	404	

非公開

Request

```
{
  "reason": "サービス方針に違反する内容が確認されたため"
}
```

Response

```
{
  "id": 125,
  "processing_status": "ready",
  "publish_status": "hidden",
  "updated_at": "2026-07-16T12:10:00Z"
}
```

公開再開

Request

```
{
  "reason": "内容を再確認し、公開可能と判断したため"
}
```

Response

```
{
  "id": 125,
  "processing_status": "ready",
  "publish_status": "published",
  "updated_at": "2026-07-16T12:30:00Z"
}
```

返してはいけないもの

- OriginalObjectKey
- VideoObjectKey
- ThumbnailObjectKey
- 元動画URL
- 固定公開URL
- Storage Credential
- SQL
- DB制約名
- Stack Trace
- Password
- PasswordHash
- Access Token
- Refresh Token
- Cookie
- HMAC鍵
- Worker内部Error Message
- FFmpegコマンド全文

hidden動画のアクセス仕様

API・画面	hidden動画
公開リール	表示しない
公開詳細	404
投稿者の投稿一覧	状態だけ表示する
投稿者の投稿詳細	再生URLを返さない
投稿者の再公開操作	拒否する
保存一覧	表示しない
後続の動画検索	表示しない
管理者投稿一覧	状態を表示する
管理者投稿詳細	期限付き変換後動画URLを返せる
Object Key	誰にも返さない

hidden動画について、一般ユーザーと投稿者へ再生URLを返さないことは要件定義で明示されている。

Frontend仕様

URL

URL	画面	認証
<code>/admin/videos</code>	管理者投稿一覧	AdminRoute
<code>/admin/videos/:video_id</code>	管理者投稿詳細	AdminRoute

AdminRoute

状態	処理
認証確認中	Loading表示
未認証	Login画面へ遷移
Roleが <code>user</code>	管理画面を表示しない
Roleが <code>admin</code>	管理画面を表示する

投稿一覧画面

表示するもの。

- 投稿者名
- タイトル
- 説明
- カテゴリー
- ProcessingStatus
- PublishStatus
- 投稿日時
- 詳細画面への導線

状態表示例：

状態	表示
uploading / private	アップロード中
expired / private	アップロード期限切れ
uploaded / private	動画処理待ち
processing / private	動画処理中
ready / published	公開中
ready / private	投稿者により非公開
ready / hidden	管理者により非公開
failed / private	動画処理失敗

保持する状態

状態	内容
items	取得済み投稿
nextCursor	次ページCursor
hasMore	次ページが存在するか
isLoading	API通信中か
error	APIエラー

次ページ取得

1. `hasMore` がtrueであることを確認する。

2. `nextCursor` が存在することを確認する。
3. `isLoading` がfalseであることを確認する。
4. APIを呼ぶ。
5. 新しいitemsを末尾へ追加する。
6. 新しいCursorを保存する。
7. `has_more` を保存する。

二重取得防止

- 通信中は追加Requestを送らない。
- 同じCursorを複数回送らない。
- Component再描画だけで再取得しない。
- `hasMore = false` の場合は取得しない。
- 再読込時はitemsとCursorを初期化する。
- FrontendでCursorをDecodeしない。

投稿詳細画面

表示するもの。

- 投稿者名
- 投稿者Status
- タイトル
- 説明
- カテゴリー
- ProcessingStatus
- PublishStatus
- 投稿日時
- 更新日時
- 確認可能な変換後動画
- 確認可能なサムネイル
- Reason入力欄

- 実行可能な管理操作

操作表示

Video状態	投稿者状態	表示操作
ready / published	activeまたはsuspended	非公開
ready / hidden	active	公開再開
ready / hidden	suspended	公開再開ボタンを無効化
ready / private	任意	操作なし
処理中	任意	操作なし
failed	任意	操作なし
deleted	任意	詳細表示なし

非公開操作

1. Reasonを入力する。
2. Reason未入力では送信しない。
3. 確認後に非公開APIを呼ぶ。
4. 送信中はボタンを無効化する。
5. 二重送信を防止する。
6. 成功後に詳細を再取得する。
7. PublishStatusを **hidden** 表示へ更新する。
8. 再生URLを管理者詳細用として再取得する。

公開再開操作

1. 投稿者Statusがactiveであることを画面表示上確認する。
2. Reasonを入力する。
3. Reason未入力では送信しない。
4. 送信中はボタンを無効化する。
5. 公開再開APIを呼ぶ。
6. 成功後に詳細を再取得する。
7. PublishStatusを **published** 表示へ更新する。

Frontendの投稿者Status判定は表示制御であり、Backendの判定を代替しない。

409時

1. 「投稿状態が既に変更されています」と表示する。
2. 詳細APIを再取得する。
3. 現在状態からボタン表示を再判定する。
4. Client側の想定状態で強制上書きしない。

XSS

Reactの通常の文字列出力を使用する。

次を行わない。

- `dangerouslySetInnerHTML`
 - TitleをHTMLとして挿入
 - DescriptionをHTMLとして挿入
 - NameをHTMLとして挿入
 - ReasonをHTMLとして挿入
 - API ErrorをHTMLとして挿入
-

OpenAPI仕様

OpenAPIへ次を記載する。

- Bearer認証必須
- activeなadmin限定
- Video ID Path Parameter
- limit
- cursor
- Reason Request Body
- 成功Response
- `400`

- 401
- 403
- 404
- 409
- 413
- 500
- Object Keyを返さないこと
- hidden 動画は一般ユーザーと投稿者へ再生URLを返さないこと
- activeな管理者投稿詳細だけ hidden 動画の期限付き再生URLを取得できること
- 非公開と監査ログ作成が同じTransactionであること
- 公開再開と監査ログ作成が同じTransactionであること
- 投稿者suspended時は公開再開できないこと

投稿一覧description

認証済みのactiveな管理者向けに、未削除の投稿を投稿日時の新しい順で取得する。処理状態や公開状態にかかわらず管理対象を返す。次ページが存在する場合はnext_cursorを返し、FrontendはCursorを変更せず次回Requestへ使用する。

投稿詳細description

認証済みのactiveな管理者向けに、未削除の投稿、投稿者、処理状態、公開状態を返す。readyかつpublishedまたはhiddenでOutputMetaが存在する場合は、管理者確認用の期限付き変換後動画URLを返す。元動画URLとObject Keyは返さない。

投稿非公開description

readyかつpublishedで未削除の投稿をhiddenへ変更する。非公開理由とRequest IDを含む監査ログを、Video状態変更と同じDB Transactionで保存する。処理中、private、hidden、failed、削除済み投稿には実行できない。

投稿公開再開description

readyかつhiddenで未削除の投稿をpublishedへ戻す。投稿者がactiveの場合だけ実行できる。公開再開理由とRequest IDを含む監査ログを、Video状態変更と同じDB Transactionで保存する。

エラー仕様

HTTP	条件
400 Bad Request	Video ID、Reason、limit、cursor、JSONが不正
401 Unauthorized	未認証、Token期限切れ、署名不正、TokenVersion不一致
403 Forbidden	一般ユーザーによる管理者API利用
404 Not Found	Videoが存在しない、または削除済み
409 Conflict	状態競合、既にhidden、既にpublished、投稿者suspended
413 Content Too Large	共通Body Limit超過
500 Internal Server Error	DB、Transaction、監査ログ、署名付きURL発行等の内部失敗

共通エラー形式

```
{
  "status":409,
  "code":"video_state_conflict",
  "message":"投稿の現在状態ではこの操作を実行できません",
  "request_id":"..."
}
```

推奨エラーコード

code	用途
validation_failed	入力不正
unauthorized	認証失敗
admin_required	管理者権限なし
video_not_found	対象なし・削除済み
video_state_conflict	状態競合
video_owner_suspended	投稿者停止中の公開再開
internal_error	内部エラー

内部のGORM Error、Storage Error、SQLSTATE、制約名をMessageへ使用しない。

セキュリティ仕様

リスク	対策
一般ユーザーの管理者操作	Auth MiddlewareとAdmin Middleware
Frontend Role改ざん	Backendで必ずRoleを再確認する
XSS	Reactの通常文字列出力
SQL Injection	GORMブレースホルダー、固定ORDER BY
状態競合	Video行ロック、現在状態付きUPDATE
監査ログ欠落	状態変更と同じTransactionでINSERT
private動画の閲覧	管理者にも期限付きURLを発行しない
hidden動画の一般閲覧	公開条件から除外し、一般・投稿者へURLを返さない
URL漏えい	短期限の署名付きURL、Object Key非公開
元動画漏えい	管理者へ元動画URLを返さない
秘密情報流出	Token、Cookie、Credential、Object KeyをResponse・Logへ出さない
二重操作	行ロックと状態再確認、Frontendボタン無効化
CSRF	Bearer Access Tokenを使用し、Refresh Cookieを管理API認証に使わない
大きなJSON	共通Body Limit

Loggerへ記録するもの

- Request ID
- 管理者User ID
- Video ID
- 操作名
- 成功・失敗
- HTTP Status
- 処理時間

Loggerへ記録しないもの

- Access Token
- Refresh Token
- Cookie
- Storage Credential
- 署名付きURL
- Object Key
- Reason全文
- Password
- PasswordHash

Reasonは監査ログに保存するが、通常のAPI Loggerへ全文を重複出力しない。

テスト仕様

Entity

- `ready / published / 未削除` をhiddenへ変更できる
- `processing`動画をhiddenへ変更できない
- `private`動画をhiddenへ変更できない
- `hidden`動画を再度hiddenへ変更できない
- `削除済み`動画をhiddenへ変更できない
- `ready / hidden / 投稿者active` をpublishedへ戻せる
- `投稿者suspended`時にrestoreできない
- `private`動画をrestoreできない
- `published`動画をrestoreできない
- `削除済み`動画をrestoreできない
- `hidden`動画を投稿者が再公開できない
- UpdatedAtが更新される
- AdminAuditLogの `video_hide` 組み合わせが有効

- AdminAuditLogの `video_restore` 組み合わせが有効
- Actionと状態が不一致のAudit Logを生成できない
- Reason空文字を拒否する
- Request ID空文字を拒否する

Repository

一覧

- 未削除Videoを取得できる
- admin投稿も一覧に含まれる
- privateを一覧に含める
- hiddenを一覧に含める
- processingを一覧に含める
- failedを一覧に含める
- 削除済みを含めない
- CreatedAt降順、ID降順になる
- Cursor境界で重複しない
- Cursor境界で欠落しない
- `limit + 1` 取得でHasMoreを判定できる
- User入力をORDER BYへ連結しない

詳細

- Videoと投稿者を取得できる
- OutputMetaを取得できる
- 削除済みはNot Foundになる
- OriginalObjectKeyをRead Modelへ公開しない

非公開

- VideoとAudit Logが同時Commitされる
- Audit Log作成失敗時にVideo変更がRollbackされる

- 行ロック後に現在状態を再確認する
- 同時非公開で一方だけ成功する
- 投稿者非公開との同時実行で不正状態にならない
- 削除との同時実行で削除済みをhiddenにしない
- SavedVideoを削除しない
- VideoLikeを削除しない

公開再開

- VideoとAudit Logが同時Commitされる
- Audit Log作成失敗時にVideo変更がRollbackされる
- 投稿者active時にpublishedへ戻る
- 投稿者suspended時に拒否する
- 同時restoreで一方だけ成功する
- User停止との同時実行後にpublishedのまま残らない
- 削除との同時実行で削除済みをpublishedにしない

Usecase

- 一般ユーザーを拒否する
- inactiveな管理者を拒否する
- 一覧CursorをUTCで生成する
- CursorをBase64 Raw URL Encodingで生成する
- 取得結果0件でCursorを生成しない
- published詳細で期限付きURLを発行する
- hidden詳細で管理者用期限付きURLを発行する
- private詳細でURLを発行しない
- Object Keyを出力へ含めない
- 非公開でReasonとRequest IDをRepositoryへ渡す
- 公開再開でReasonとRequest IDをRepositoryへ渡す
- 投稿者停止中に公開再開できない

Validator

- Video IDが数字
- Video IDが1以上
- Video IDが`math.MaxInt64` 以下
- Reasonをtrimする
- Reason空文字を拒否する
- Reason 500文字境界
- Reason 501文字を拒否する
- limit未指定で20
- limit 1を許可する
- limit 100を許可する
- limit 0を拒否する
- limit 101を拒否する
- 正しいCursorをDecodeできる
- 不正Base64を拒否する
- 不正JSONを拒否する
- 不明Cursorフィールドを拒否する
- 不正日時を拒否する
- ID 0を拒否する

Controller

- 一覧で200
- 詳細で200
- 非公開で200
- 公開再開で200
- 不正JSONで400
- 不正Video IDで400
- Reasonなしで400

- 未認証で401
- 一般ユーザーで403
- 存在しないVideoで404
- 状態競合で409
- 投稿者suspendedで409
- Body Limit超過で413
- 内部エラーで500
- Error ResponseへRequest IDを含める
- Object KeyをResponseへ含めない
- SQLやStack TraceをResponseへ含めない

Middleware

- 未認証を拒否する
- 不正Tokenを拒否する
- TokenVersion不一致を拒否する
- 一般ユーザーを403で拒否する
- activeなadminを通す
- Controller到達前に一般ユーザーを拒否する

Frontend

- 一覧を表示できる
- 空配列を正常表示できる
- 次Cursorで追加取得できる
- 同じCursorを二重送信しない
- hasMoreがfalseなら追加取得しない
- 詳細を表示できる
- hidden動画を管理者詳細で再生できる
- private動画へ再生UIを表示しない
- published時だけ非公開ボタンを表示する

- hiddenかつ投稿者active時だけ公開再開ボタンを有効化する
 - 投稿者suspended時は公開再開を無効化する
 - Reason未入力では送信しない
 - 送信中は二重送信しない
 - 409時に詳細を再取得する
 - HTMLを実行しない
 - Object Keyを画面へ表示しない
-

API結合テスト

投稿一覧・詳細

1. adminでログインする。
2. 一般ユーザーで複数状態の動画を作成する。
3. `GET /admin/videos` を呼ぶ。
4. 未削除動画が新しい順で返る。
5. private、published、hidden、processing、failedが管理者一覧へ含まれる。
6. 削除済み動画が含まれない。
7. 詳細APIを呼ぶ。
8. 投稿者、タイトル、説明、カテゴリー、状態が返る。
9. Object Keyが返らない。

非公開

1. `ready / published` 動画を用意する。
2. adminで非公開APIを呼ぶ。
3. Videoが `ready / hidden` になる。
4. `video_hide` の監査ログが1件作成される。
5. AdminUserIDが操作管理者になる。
6. TargetIDがVideo IDになる。

7. ReasonとRequest IDが保存される。
8. リールから消える。
9. 保存一覧から消える。
10. 投稿者詳細APIで再生URLが返らない。
11. 投稿者が再公開しようとするすると拒否される。
12. 管理者詳細APIでは確認用URLが返る。

公開再開

1. `ready / hidden` 動画を用意する。
2. 投稿者をactiveにする。
3. adminで公開再開APIを呼ぶ。
4. Videoが `ready / published` になる。
5. `video_restore` の監査ログが1件作成される。
6. リールへ再表示される。
7. 保存関係が残っている場合は保存一覧へ再表示される。

停止User

1. User停止により `video_hide_by_user_suspension` でhiddenとなった動画を用意する。
2. 停止中に公開再開APIを呼ぶ。
3. `409 Conflict` になる。
4. Userを再開する。
5. 動画が自動公開されないことを確認する。
6. 管理者が公開再開APIを呼ぶ。
7. publishedへ戻る。
8. `video_restore` 監査ログが作成される。

権限

1. 一般ユーザーでログインする。
2. 管理者投稿一覧APIを呼ぶ。

3. `403 Forbidden` になる。
 4. Controllerへ到達しない。
 5. Frontendで管理者画面を直接開いても表示されない。
-

ブラウザ確認

管理者投稿一覧

1. adminでログインする。
2. 管理者投稿一覧を開く。
3. 投稿者名が表示される。
4. タイトルと説明が表示される。
5. 処理状態が表示される。
6. 公開状態が表示される。
7. 投稿日時が表示される。
8. 次ページを取得する。
9. 重複投稿が表示されない。
10. 削除済み投稿が表示されない。

投稿非公開

1. 公開中の投稿詳細を開く。
2. 変換後動画を確認する。
3. Reasonを入力する。
4. 非公開を実行する。
5. `hidden` 表示へ変わる。
6. リールから消える。
7. 投稿者の画面で動画を再生できない。
8. 投稿者に再公開ボタンが表示されない。
9. 管理者詳細では確認用動画を再生できる。

投稿公開再開

1. hidden動画を開く。
2. 投稿者Statusがactiveであることを確認する。
3. Reasonを入力する。
4. 公開再開を実行する。
5. **published** 表示へ変わる。
6. リールへ再表示される。

停止中の投稿者

1. 投稿者をsuspendedにする。
2. hidden動画を開く。
3. 公開再開ボタンが無効であることを確認する。
4. APIを直接呼んでも409になることを確認する。
5. 投稿者をactiveへ戻す。
6. 動画が自動公開されないことを確認する。
7. 管理者が個別に公開再開できることを確認する。